

Supplementary Material

I. NOTATION

One symbol, one meaning. The symbols α, β, γ and R_{safe} appeared in an earlier draft and are retired.

Symbol	Meaning
δ	conformal risk-control level (target $\mathbb{E}[L] \leq \delta$)
$\tau, \hat{\tau}$	admissibility threshold; $\hat{\tau}$ calibrated, then frozen
U_{max}	bound on predicted uncertainty width
ηJ	replay dispersion band, a property of one runtime stack
$\lambda_F, \lambda_L, \lambda_C$	objective weights in J
w_L, w_R, w_B, w_D	cost-component weights in $C(a)$
Δ_A	availability non-inferiority margin
P_{ID}	the run-generating distribution the guarantee is stated for
B	bootstrap replicas of the structural model
H	rollout steps, $\Delta/\Delta_{\text{tick}}$

II. THE CRC PROCEDURE, EXACTLY

Implemented in `intelligence/conformal/crc.py`;
tested in `intelligence/tests/test_crc.py`.

- 1) For calibration run i and threshold τ_j from a fixed grid, compute the within-run set-level loss $L_i(\tau_j) = |\mathcal{T}_i|^{-1} \sum_{t \in \mathcal{T}_i} \mathbf{1}[\exists a \in \mathcal{A}_{it}^{\text{safe}}(\tau_j) : Y_{it}^a = 1]$.
- 2) $\hat{L}_n(\tau_j) = n^{-1} \sum_i L_i(\tau_j)$.
- 3) Corrected risk $(n\hat{L}_n(\tau_j) + B_\ell)/(n+1)$ with $B_\ell = 1$.
- 4) $\hat{\tau} = \max\{\tau_j : \text{corrected}(\tau_j) \leq \delta\}$ — the *largest* admissible threshold, since $\mathcal{A}^{\text{safe}}$ grows with τ and the widest admissible set leaves the cost rule the most room.
- 5) If no τ_j qualifies, $\hat{\tau}$ is undefined: the controller abstains everywhere and that is reported, rather than the level being relaxed until something passes.

The correction is not optional at the sample sizes used here. At $n = 20$ calibration runs, $B_\ell/(n+1) \approx 0.048$ — the same order as δ itself. A unit test pins this value so it cannot quietly shrink, and another verifies empirically, over 200 trials, that the realised loss on held-out runs stays at or below δ .

A. Assumptions, and where each one fails

- *Monotone bounded loss.* $L_i \in [0, 1]$ and is non-decreasing in τ because relaxing the threshold only enlarges the admissible set. This is the property that makes CRC applicable at all; a loss defined on the *selected* action would not have it, since a cheaper admitted action can fail where a costlier one would not.
- *Exchangeability of runs.* Calibration and identically distributed test runs are drawn independently from P_{ID} (§III) with disjoint seed blocks. Independent seeds alone do not deliver exchangeability: the runs must come from the same declared generator.
- *Scenario shift.* F5–F7 lie outside P_{ID} by construction. There $\hat{\tau}$ is carried over unchanged, no guarantee is claimed, and realised risk is reported empirically.

- *The fallback is not covered.* When $\mathcal{A}^{\text{safe}}(\hat{\tau}) = \emptyset$ the loss is zero by construction whatever the fallback then does. Fallback rate, fallback failure rate and fallback agreement-violation rate are reported separately.
- *No online recalibration.* Updating the set-level loss needs outcomes for actions the running system did not take. A deployed edge system cannot observe those; only a reconstructible environment can.

III. THE ID RUN-GENERATING DISTRIBUTION P_{ID}

Declared in `experiments/configs/p_id.yaml`: scenario uniform over $\{\text{F1}, \dots, \text{F4}\}$; severity sampled from a per-scenario range; workload device count and event rate from declared ranges; seed drawn from a block disjoint by split (train, validation, calibration, ID test). Anchors are taken on a pre-declared grid *after* the start-up transient — the first `warmup_s` plus the first 30s of measurement — because early anchors sit in a queue-fill transient where every action looks alike. That was observed during M2 and is a selection rule, never an outcome-dependent choice.

IV. UNCERTAINTY

$\hat{U}(a \mid S_\ell) = Q_{0.95}\{\hat{R}_b\} - Q_{0.05}\{\hat{R}_b\}$ over B bootstrap replicas of the structural model: a quantile width rather than a standard deviation, so a few divergent replicas do not dominate the filter. A 5–95 interval from a small ensemble rests on its extreme order statistics, so B is selected in the pilot from $\{20, 30, 50\}$ on the stability of $\hat{U}$ and of the induced action ranking against inference overhead — on validation data only. U_{max} is likewise validation-only and frozen before calibration begins.

$\hat{U}$ *filters*; it is not itself calibrated. CRC calibrates $\hat{\tau}$. The correct description of the result is an *uncertainty-filtered, risk-calibrated admissible set*.

V. COST NORMALISATION

$\tilde{x} = \text{clip}((x - x_{\min}^{\text{cal}})/(x_{\max}^{\text{cal}} - x_{\min}^{\text{cal}}), 0, 1)$ per component, with ranges taken from the calibration split alone. Two cases need naming rather than silent handling: *degenerate range* — if $x_{\max}^{\text{cal}} = x_{\min}^{\text{cal}}$ the component is constant on calibration and contributes $\tilde{x} = 0$, and the occurrence is logged; *out-of-range under shift* — a component may exceed the calibration range on F5–F7, and clipping keeps the cost in $[0, 1]$. Without it an unclipped value above one would silently reweight the objective on exactly the runs where the method is under most strain. The clip rate is reported per scenario.

Equal weights $w = 0.25$ are a neutral a-priori default, fixed in version control before any comparative result is examined. No mathematical conservatism follows from equality, and claiming it would be rhetoric. Sensitivity to alternative operational preferences is in §VIII.

VI. REPLAY: ANCHORS, FINGERPRINTS AND QUIESCENCE

A. Two exact gates

A branch set proceeds only when *both* match the anchor exactly.

StructuralStateFingerprint — logical tick; configuration hash; routing assignment; rate limits *and* current token-bucket levels; per-queue content digest with head and tail event ids; retry pending-set digest, attempt counts and backoff deadlines in logical time; timeout deadlines; service placement; replica state; pending migrations; edge capacity; per-producer sequence positions; processed counters; fault phase; action history.

TransportQuiescenceFingerprint — per-producer end-of-epoch sequence declarations and per-consumer drain confirmations.

RuntimeObservationFingerprint — CPU, RSS, wall-clock latency, scheduler and socket timings. Expected to vary; recorded as evidence; never a gate.

No tolerance is admitted on the first two. Two queues of equal depth holding different events have different futures, and a tolerance would let pre-action state mismatch enter η_J , making the resolution band partly a measure of its own imprecision.

B. Why local acknowledgement is insufficient

On an asynchronous bus a component can truthfully report finishing tick $k - 1$ while a message it published during that tick is still in a socket buffer. Both branches would then show identical local counters and differ in what is in flight — invisible to local bookkeeping, and presenting as a valid common anchor. Each producer therefore declares $\text{END_EPOCH}(k, p, \text{last_seq})$ and each consumer confirms $\text{DRAINED}(k, p, c, \text{consumed_through})$; the anchor opens only when every consumer has consumed every declaring producer through its declared last sequence. A timeout never proceeds: it reports which pair is outstanding and by how much.

The protocol establishes completeness, not determinism. Within an epoch, messages are delivered and processed in real arrival order — reordering them would erase the nondeterminism the audit exists to measure.

C. D0 and the resolution band

$\eta_J = Q_{0.95}(|J_{r_i}^{\text{obs}}(a) - J_{r_j}^{\text{obs}}(a)|)$ over repeat pairs of the same anchor under the same action. The $\binom{30}{2} = 435$ pairwise differences from 30 repeats are *not* independent, so η_J is reported as an operational resolution and any interval for it is obtained by resampling *anchors*, never by treating pairwise differences as independent observations.

D0-lite (10 anchors $\times$ 3 actions $\times$ 20 repeats) precedes the full audit. Its pre-declared criterion is

$$\text{SNR}_J = \frac{\text{median}_{t, a_i \neq a_j} |\text{median } J_t(a_i) - \text{median } J_t(a_j)|}{\eta_J} \geq 3,$$

reported alongside $P_{\text{resolvable}} = \Pr(|\Delta J| > \eta_J)$, because a median can hide the shape of the distribution when some

action pairs are naturally equivalent at some anchors. The threshold is not revised after seeing the number. Below it the project stops and repairs replay fidelity rather than implementing further actions or training anything.

η_J belongs to one runtime stack. The manifest records `runtime_stack_id` over Go version, kernel, bus type and version, transport configuration hash, RPC type, container image digests, netem configuration and process topology. Changing the transport invalidates D0.

VII. EXPERIMENTAL PROTOCOL

A. Scenarios

[TBD — to be measured, not estimated] parameters as declared in `experiments/scenarios/F*.yaml`; F1–F4 constitute P_{ID} ; F5–F7 are held out entirely.

B. Splits

Training, validation and calibration draw disjoint seed blocks from F1–F4. The ID test set uses new seeds *and* new parameter values — new seeds alone would measure only seed variance. Replay-derived records are quarantined from every training set and a leakage test fails the build if one appears.

C. Interventional training data

At every training anchor all candidate actions are executed in their own branches, giving (S_t, a, S_{t+1}^a, Y^a) for all a : assignment is forced rather than sampled, which is a designed experiment on the emulator and removes the need for propensity weighting. Each branch records whether the action mutated runtime state; a dispatched action that changed nothing is a relabelled no-op and is excluded rather than averaged in.

D. Baseline fairness

B1 has the same action primitives as CSC — a baseline with a poorer repertoire would let CSC win on options rather than on decision quality. What it lacks is a model: it maps observable symptoms to responses by fixed priority, with hysteresis on entry and release. It may not read the scenario label or fault-injector state. Its thresholds are frozen on pilot/validation data before any comparative run. All learned controllers share features, observation window, action set, execution mechanism and hyperparameter budget; none sees future information or replay outcomes.

E. Statistics

95% bootstrap intervals over 10,000 resamples; paired Wilcoxon signed-rank; Cliff’s delta; Holm correction across the primary family. H3 is a one-sided non-inferiority claim at $\Delta_A = 0.005$, not an inference from a non-significant difference; 0.005 is the largest availability reduction treated as practically negligible over the 300 s measurement window, about 1.5 s of additional unavailability. H4 is a tail claim: $p_{95}(T_{\text{dec}}) < 100 \text{ ms}$ up to 10,000 devices with a deadline-miss rate below 1%.

F. Exclusions

Documented technical failures only — out-of-memory kills, orchestration crashes before the measurement window closes, a fault injector reporting an unapplied schedule. Logged with evidence, reported as counts, under a policy fixed before results were examined. No run is excluded on the basis of its outcome.

VIII. SENSITIVITY ANALYSES

All secondary; the pre-registered primary settings are unchanged by them.

- $\delta \in \{0.05, 0.10, 0.20\}$ — gate permissiveness against abstention.
- $\Delta_A \in \{0.001, 0.0025, 0.005, 0.01\}$ — primary claim at 0.005 only.
- Non-uniform cost weights.
- $B \in \{20, 30, 50\}$ — ensemble stability against inference overhead.
- Horizon $\Delta \in \{2, 5, 10\}$ s — selected on validation, then frozen.

IX. HYPERPARAMETERS AND ENVIRONMENT

[TBD — to be measured, not estimated] Model architecture and training settings, the frozen $\hat{\tau}$, $U_{\max}$, B , Δ , H and η_J with its `runtime_stack_id`, plus software versions and host description, are filled from the run manifests once the reportable stack is validated. They are left empty rather than guessed.

X. REPRODUCTION

```
make build && ./scripts/topology_smoke.sh # no Docker needed
make deps-adapters && make local-validate # Alb gate, Linux + root
docker compose up --build                # full topology
make replay                             # branch mechanics
make reproduce                           # full pipeline
```

Runs produced without kernel-level impairment record `impairment_mode=application_layer` and are excluded from reported comparisons. `check_reportable_stack.py` refuses to start a reportable runner on a development substrate even if the validated flag is set by hand.